24-Week Parallel Study Plan

For **Antonio Vasconcelos** | Python · AI · AWS · Blockchain · English · LeetCode | 2 hours a day: **English 8:00–9:00 AM** + **Tech 2:00–3:00 PM** (America/Fortaleza) | Mon Oct 12, 2026 → Sun Mar 28, 2027

> **How this plan works:** there are **no sequential phases**. You study **every topic every week**, in parallel, in two fixed daily sessions: a morning **English** hour and an afternoon **tech** hour. Each topic moves through three levels at the same time: **Foundations** (months 1–2), **Intermediate** (months 3–4) and **Advanced / projects** (months 5–6). Section 4 shows the big picture; **Section 5 lists the exact subject of every session, week by week**, for all 24 weeks.

1. Every Day: Two Sessions

Session	Time	Block	What to do
Morning	8:00 – 9:00 AM	**English** (60 min)	The structured routine in Section 2, built around the week's grammar point and vocabulary theme (Section 5).
Afternoon	2:00 – 2:45 PM	**Main topic** (45 min)	Today's tech subject from Section 5. About 10 min reading/video, 35 min hands-on coding or a lab.
	2:45 – 3:00 PM	**LeetCode** (15 min)	One problem from the week's NeetCode 150 category (Section 5). Stuck after 15 min? Read the solution, note the pattern, re-solve it on Sunday.

2. Morning English Routine: 8:00–9:00 AM

Time	Block	Mon – Sat	Sunday (weekly review)
8:00 – 8:10	**Listening** (10 min)	One BBC *6 Minute English* episode or a short tech talk on the week's theme: once without subtitles, once with. Note 2 useful phrases.	Re-listen to the week's favourite episode without subtitles.
8:10 – 8:25	**Grammar + reading** (15 min)	**Mon:** learn the week's grammar point (British Council lesson + exercises). **Tue–Sat:** read a short article or docs page about your tech topic; underline examples of the grammar point and pick 2 new words.	Redo the grammar exercises without notes; check your mistakes.
8:25 – 8:40	**Speaking / shadowing** (15 min)	Shadow 1–2 minutes of the listening audio (repeat right after the speaker, copying rhythm and stress), then speak for 2 minutes about yesterday's tech session using the grammar point.	Record a **2-minute talk** on the week's theme; listen back and note 3 fixes.
8:40 – 8:50	**Writing** (10 min)	Write 5–6 sentences with the grammar point and the theme words (e.g. a stand-up update or a notes.md entry). Check them with Write & Improve.	Write an 8–10-sentence weekly summary of everything you studied.
8:50 – 9:00	**Flashcards** (10 min)	Review your Anki deck; add today's 2 new words with an example sentence each (~12 words a week).	Clear all due cards; tag the hardest 5 for extra review.

3. The Fixed Weekly Tech Routine: 2:00–3:00 PM

Day	2:00–2:45 Main topic (45 min)	2:45–3:00 LeetCode (15 min)
Mon	**Python**	New problem
Tue	**AWS**	New problem
Wed	**AI**	New problem
Thu	**Blockchain**	New problem
Fri	**Python** odd weeks / **AI** even weeks	New problem
Sat	**AWS** odd weeks / **Blockchain** even weeks	New problem
Sun	**Review + mini project** combine what you learned this week	Re-solve 1 problem from the week

Balance check	Python	AI	AWS	Blockchain	LeetCode	English
Odd week (1, 3, 5 …)	2 sessions	1	2	1	7 × 15 min every day	7 × 60 min every morning
Even week (2, 4, 6 …)	1	2	1	2		
Every 2 weeks	**3**	**3**	**3**	**3**	**14** problems	**14** hours

Week 1 starts Mon Oct 12, 2026, so it is an odd week. Every tech topic gets exactly the same time over any two weeks.

Consistency rules: missed a session? Don't double up; just continue with the next one. Holiday weeks (W11 with Dec 24–25, W12 with Dec 31–Jan 1) can be lighter. Keep one *notes.md* per topic (in English) and push all code to GitHub. The Cloud Practitioner exam itself (~90 min) is booked outside the study hour.

Checkpoints (all tracks together):
End of Week 8 (Sun Dec 6): Foundations done in every track; first milestone project for each topic on GitHub; ~90 English words.
End of Week 16 (Sun Jan 31): Intermediate done; API deployed on AWS; Cloud Practitioner exam taken or booked; ~180 English words.
End of Week 24 (Sun Mar 28): capstone projects shipped; ~150 LeetCode problems practiced; 270+ English words and 24 grammar points; capstone demo recorded in English.

4. Parallel Roadmap: All Tracks at the Same Time

Read the grid by columns: during each period you work on all six rows at once, each in its own slot of the routine. Section 5 breaks every cell down into weekly subjects.

Track	Months 1–2 · Foundations Weeks 1–8 · Oct 12 – Dec 6	Months 3–4 · Intermediate Weeks 9–16 · Dec 7 – Jan 31	Months 5–6 · Advanced / projects Weeks 17–24 · Feb 1 – Mar 28
Python	Type hints in depth (*typing*, Protocol, TypedDict, *mypy*); dataclasses vs. Pydantic; decorators (*functools.wraps*); iterators & generators; context managers	Async: event loop, *async/await*, *asyncio.gather*, timeouts, *httpx*; async API endpoints; *pytest* (fixtures, parametrize, mocking, coverage)	Packaging (*pyproject.toml*, TestPyPI); profiling and performance; clean project structure; Python as the glue for the AI, AWS and web3 projects
AI	ML fundamentals: supervised vs. unsupervised, train/test split, linear & logistic regression; NumPy/pandas refresher; scikit-learn classification and metrics	Neural network intuition (loss, gradient descent, backprop); a tiny PyTorch network; transfer learning (fast.ai lessons 1–2); LLM basics: prompting, JSON output, embeddings	RAG: chunking, vector store (FAISS or Chroma), retrieval + generation, evaluation; agents: tool calling, agent loop, guardrails, cost/latency; ship the app
AWS	Cloud concepts, regions/AZs; **billing budget alert on day 1**; IAM (roles, policies, MFA, least privilege); AWS CLI; EC2 and VPC basics; S3 + *boto3*	Lambda + API Gateway (serverless API), CloudWatch logs; DynamoDB vs. RDS; deploy your API; **Cloud Practitioner exam prep** and exam (aim for late January)	Infrastructure as code basics (CloudFormation or AWS SAM); AI on AWS (e.g. Amazon Bedrock); secrets, monitoring and cost control for your deployed apps
Blockchain	Hashing, blocks, consensus (PoW vs. PoS), keys and wallets, transactions; Ethereum accounts, gas, the EVM (ethereum.org); start CryptoZombies	Solidity with CryptoZombies + Remix IDE: types, functions, mappings, structs, events, modifiers; write and test your first contract	Deploy to a testnet (e.g. Sepolia); security basics (reentrancy, access control); *web3.py*: read state, send transactions, listen to events
English	Daily 1-hour morning session. Grammar: present simple vs. continuous, articles, nouns & quantifiers, prepositions, comparatives, future forms, modals, questions. Themes: your job, tools, data, meetings. Write all notes and commit messages in English	Past simple, present perfect (simple & continuous), narrative tenses, conditionals 0–2, gerunds vs. infinitives, relative clauses. Themes: bugs, stand-ups, trade-offs, cloud verbs. Write project READMEs in English	Passive voice (common in tech docs), linking words, reported speech, phrasal verbs, polite language. Themes: docs, changelogs, architecture, interviews, demos. Record a 2-minute demo in English
LeetCode	NeetCode 150: Arrays & Hashing, Two Pointers, Sliding Window, Stack. Target: easy in under 15 min	Binary Search, Linked List, Trees, Heap / Priority Queue. Target: easy → first mediums	Backtracking, Graphs, 1-D Dynamic Programming, Intervals, Greedy + review of weak spots. Target: mostly medium

5. Week-by-Week Plan: Every Session, All 24 Weeks

Tech hour 2:00–3:00 PM (45 min subject + 15 min LeetCode) and English hour 8:00–9:00 AM. Fri/Sat: Py Python + AWS in odd weeks, AI AI + BC Blockchain in even weeks. LeetCode: ~6 new problems from the category (examples listed) + 1 Sunday re-solve. English: **G** = grammar point, **V** = vocabulary theme.

Week	Mon Python	Tue AWS	Wed AI	Thu Blockchain	Fri Py / AI	Sat AWS / BC	Sun Review + mini project	LeetCode focus (2:45–3:00)	English focus (8–9 AM)
Months 1–2 · Foundations · Weeks 1–8									
W1 Oct 12–18	Type hints basics: annotate functions, *list/dict/Optional*; run *mypy*	Account set-up: root MFA, **billing budget alert**, Free Tier limits	What ML is: supervised vs. unsupervised; the ML workflow	What a blockchain is; hash functions (SHA-256 with *hashlib*)	Py Generics, *TypeVar*, *Callable*; run mypy on one of your APIs	AWS Cloud concepts: regions, AZs, shared responsibility model	Typed Python script that SHA-256-hashes every file in a folder	Arrays & Hashing I: Contains Duplicate, Valid Anagram, Two Sum	**G:** Present simple vs. present continuous **V:** Introducing yourself and your job role
W2 Oct 19–25	*Protocol* (structural typing) and *TypedDict*	IAM users, groups and policies; least privilege	NumPy refresher: arrays, broadcasting, vectorization	Blocks and chaining; Merkle trees	AI pandas refresher: DataFrames, cleaning, *groupby*	BC Toy blockchain in Python, part 1: *Block* class + hash links	TypedDict config + pandas summary report of a CSV	Arrays & Hashing II: Top K Frequent, Product of Array Except Self, Longest Consecutive Sequence	**G:** Articles: *a / an / the* / no article **V:** Software tools and the dev environment
W3 Oct 26 – Nov 1	*dataclasses*: *field*, *frozen*, *slots*	IAM roles and MFA; AWS CLI profiles (IAM Identity Center)	Train/test split, overfitting, cross-validation	Consensus: proof of work vs. proof of stake	Py Pydantic models and validation; dataclasses vs. Pydantic	AWS AWS CLI practice: *sts*, *iam*, *s3* commands	Pydantic model that validates dataset rows before a train/test split	Two Pointers I: Valid Palindrome, Two Sum II, 3Sum	**G:** Countable / uncountable nouns; *much, many, a few, a little* **V:** Data, files and storage
W4 Nov 2–8	Decorators: closures, *functools.wraps*	S3: buckets, storage classes, versioning, bucket policies	Linear regression: intuition + scikit-learn	Toy blockchain, part 2: simple proof of work + chain validation	AI Logistic regression for classification	BC Public/private keys, digital signatures, wallets	A *@timed* decorator measuring your toy blockchain's mining time	Two Pointers II: Container With Most Water, Trapping Rain Water + 1 re-solve	**G:** Prepositions of time and place: *in / on / at* **V:** Meetings and scheduling
W5 Nov 9–15	Decorators with arguments; build a *@retry* decorator	*boto3*: upload, list, download S3 objects	Classification metrics: accuracy, precision, recall, F1, confusion matrix	Transactions; UTXO vs. account model	Py Typing decorators with *ParamSpec*; class decorators	AWS VPC basics: subnets, route tables, security groups	*boto3* uploader with your *@retry* decorator	Sliding Window I: Best Time to Buy/Sell Stock, Longest Substring Without Repeating Chars, Character Replacement	**G:** Comparatives and superlatives **V:** Describing performance: fast, reliable, scalable
W6 Nov 16–22	Iterators: the iterator protocol, *__iter__/__next__*, *itertools*	EC2: launch an instance, key pairs, instance types, SSH	Decision trees and random forests	Ethereum accounts (EOA vs. contract) and ether	AI Feature engineering + scikit-learn *Pipeline* (scaling, one-hot)	BC Gas, fees and the EVM	Run your scikit-learn notebook on an EC2 instance	Sliding Window II: Permutation in String, Minimum Window Substring, Sliding Window Maximum	**G:** Future forms: *will* vs. *going to* vs. present continuous **V:** Planning, estimates and deadlines
W7 Nov 23–29	Generators: *yield*, generator expressions, *yield from*	Deploy your API on EC2 with a least-privilege IAM instance role	Unsupervised learning: k-means clustering, PCA	Smart contract concepts (ethereum.org developer intro)	Py Generator-based data export for your API (milestone piece)	AWS EBS, AMIs, stop vs. terminate; cost check	Stream a generator export straight to S3	Stack I: Valid Parentheses, Min Stack, Evaluate Reverse Polish Notation	**G:** Modals: *can, must, have to, should* **V:** Requirements, rules and permissions
W8 Nov 30 – Dec 6	Context managers: *with*, *@contextmanager* (contextlib); mypy-clean wrap-up	Milestone: *boto3* S3 manager script; delete unused resources	Milestone: classifier on a public dataset (e.g. UCI)	CryptoZombies lesson 1, part 1	AI Metrics report + README for the classifier	BC CryptoZombies lesson 1 finished + toy blockchain README	**Checkpoint 1:** publish all Foundations milestone repos	Stack II: Generate Parentheses, Daily Temperatures, Car Fleet	**G:** Questions: *do/does*, wh-questions, indirect questions + review of weeks 1–7 **V:** Asking for help and code review

Week	Mon Python	Tue AWS	Wed AI	Thu Blockchain	Fri Py / AI	Sat AWS / BC	Sun Review + mini project	LeetCode focus (2:45–3:00)	English focus (8–9 AM)
Months 3–4 · Intermediate · Weeks 9–16									
W9 Dec 7–13	asyncio basics: event loop, coroutines, *async/await*	Lambda basics: handler, runtime, execution role	Neural network intuition: neurons, layers, activations (3Blue1Brown ch. 1)	Remix IDE tour; Solidity types and state variables	Py *asyncio.gather*, tasks, timeouts (*asyncio.timeout*)	AWS API Gateway + Lambda: your first serverless endpoint	Async script fetching 10 URLs with *gather* + timeouts	Binary Search I: Binary Search, Search a 2D Matrix, Koko Eating Bananas	**G:** Past simple (regular and irregular verbs) **V:** Bugs and incidents: crash, fix, roll back
W10 Dec 14–20	*httpx.AsyncClient*: concurrent HTTP calls	CloudWatch logs, metrics and alarms	Loss functions and gradient descent (3Blue1Brown ch. 2)	Functions, visibility, *view*/*pure*	AI Backpropagation intuition (3Blue1Brown ch. 3–4)	BC CryptoZombies lesson 2: mappings and *msg.sender*	Async *httpx* client that calls your API Gateway endpoint	Binary Search II: Min in Rotated Sorted Array, Search in Rotated Sorted Array, Time Based Key-Value Store	**G:** Present perfect vs. past simple **V:** Progress updates and stand-ups
W11 Dec 21–27	Async endpoints in your API (e.g. FastAPI *async def*)	DynamoDB: tables, partition/sort keys, *boto3* CRUD	PyTorch basics: tensors and autograd	Structs and arrays in Solidity	Py Async pitfalls: blocking calls, *run_in_executor*	AWS RDS vs. DynamoDB: when to use which	Holiday week (Dec 24–25): light review; sketch a "study log" struct	Linked List I: Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle	**G:** Present perfect continuous; *for* vs. *since* **V:** Small talk and holidays
W12 Dec 28 – Jan 3	pytest basics: discovery, asserts, fixtures	Deploy your API on Lambda + API Gateway + DynamoDB	Tiny PyTorch network: a training loop on MNIST	Events and modifiers (CryptoZombies lesson 3)	AI fast.ai lesson 1: transfer learning for image classification	BC *require*, custom errors, *payable* functions	Year-end review (Dec 31–Jan 1): pytest tests for the toy blockchain	Linked List II: Reorder List, Remove Nth Node From End, Add Two Numbers	**G:** Past continuous and narrative tenses **V:** Telling the story of your year; goals
W13 Jan 4–10	pytest *parametrize* marker; testing edge cases	Cloud Practitioner prep: cloud concepts + global infrastructure (Skill Builder)	fast.ai lesson 2: train on your own data, export the model	Design your contract (tip jar or study log): state, functions, events	Py Fixture scopes and *conftest.py*	AWS Architecture diagram of your API; Well-Architected pillars	Parametrized tests for all your API endpoints	Trees I: Invert Binary Tree, Maximum Depth, Diameter of Binary Tree	**G:** Zero and first conditional **V:** Logic and control flow: *if the request fails...*
W14 Jan 11–17	Mocking: *unittest.mock*, *monkeypatch*	Cloud Practitioner prep: security & compliance (IAM, KMS, Shield)	LLM basics: tokens, context window, prompting techniques	Write the contract in Remix	AI Call an LLM API from Python: system prompt, temperature	BC Deploy and test the contract in the Remix VM	Mock *boto3* calls in your tests	Trees II: Same Tree, Subtree of Another Tree, Lowest Common Ancestor of a BST	**G:** Second conditional **V:** Trade-offs and hypotheticals
W15 Jan 18–24	Testing async code (*pytest-asyncio*) and endpoints	Cloud Practitioner prep: billing & pricing (pricing models, Cost Explorer, Support plans)	Structured JSON output validated with Pydantic	Unit tests for the contract (Remix tests or a Foundry intro)	Py Coverage with *pytest-cov*; reach 80%+	AWS **Full practice exam #1**; review every mistake	LLM JSON output saved to DynamoDB	Trees III: Level Order Traversal, Validate BST, Kth Smallest in a BST	**G:** Gerunds vs. infinitives (*enjoy doing*, *decide to do*) **V:** Cloud and DevOps verbs: deploy, provision, scale
W16 Jan 25–31	Milestone: async endpoints + test suite running in GitHub Actions	Practice exam #2; **take the Cloud Practitioner exam** (or book it)	Embeddings: what they are, cosine similarity	Milestone: contract + tests on GitHub	AI Milestone wrap-up: JSON LLM script + PyTorch/fast.ai repo	BC Gas optimization basics	**Checkpoint 2:** API deployed, exam taken, Intermediate repos published	Heap / Priority Queue: Kth Largest in a Stream, Last Stone Weight, K Closest Points to Origin	**G:** Relative clauses: *who / which / that / where* + review of weeks 9–15 **V:** Defining technical terms (your exam glossary)

Week	Mon Python	Tue AWS	Wed AI	Thu Blockchain	Fri Py / AI	Sat AWS / BC	Sun Review + mini project	LeetCode focus (2:45–3:00)	English focus (8–9 AM)
Months 5-6 · Advanced / projects · Weeks 17–24									
W17 Feb 1–7	Project structure: *src/* layout, modules, config via env vars	Infrastructure as code; CloudFormation template basics	RAG architecture; load and chunk documents	MetaMask wallet + Sepolia faucet; how testnets work	Py *pyproject.toml*: build backend, dependencies, entry points	AWS AWS SAM: define Lambda + API in *template.yaml*; *sam deploy*	Python CLI that chunks your *notes.md* files for RAG	Backtracking I: Subsets, Combination Sum, Permutations	**G:** Passive voice: present simple **V:** Documentation language
W18 Feb 8–14	Build and publish your utility library to TestPyPI	SAM: add a DynamoDB table and IAM policies to the template	Embed the chunks and store them in Chroma or FAISS	Deploy the contract to Sepolia; verify it on Etherscan	AI Retrieval: top-k search, metadata filters	BC Security: reentrancy and checks-effects-interactions	Install your own library from TestPyPI; build the vector index	Backtracking II: Subsets II, Combination Sum II, Word Search	**G:** Passive voice: past simple and present perfect **V:** Release notes and changelogs
W19 Feb 15–21	Profiling: *cProfile*, *timeit*	Secrets Manager / Parameter Store for API keys	Generation: prompt with retrieved context, add citations	Security: access control (*Ownable*) and common pitfalls	Py Performance: data structures, *lru_cache*, avoiding N+1 calls	AWS Amazon Bedrock: model access; invoke a model with *boto3*	RAG answer over your notes using Bedrock	Graphs I: Number of Islands, Clone Graph, Rotting Oranges	**G:** Linking words for contrast: *however, although, whereas* **V:** Comparing technologies
W20 Feb 22–28	Logging and error-handling patterns for production	Bedrock embeddings; Knowledge Bases overview	RAG evaluation: test questions, retrieval hit rate, answer quality	web3.py: connect to an RPC node, read balances and contract state	AI Improve RAG: chunk size, overlap, re-ranking	BC web3.py: call your contract's view functions with the ABI	Profile the RAG pipeline with *cProfile*	Graphs II: Pacific Atlantic Water Flow, Course Schedule, Course Schedule II	**G:** Linking words for cause and result: *because, therefore, as a result* **V:** Explaining architecture decisions
W21 Mar 1–7	Python service layer for the RAG app (AI capstone)	Deploy the AI capstone with SAM (Lambda + API Gateway)	Agents: tool-calling basics	web3.py: sign and send transactions	Py CLI with *argparse* or *typer* for the web3 project	AWS Monitoring: CloudWatch dashboards and alarms, X-Ray basics	Agent tool that reads your smart contract via web3.py	1-D DP I: Climbing Stairs, House Robber, House Robber II	**G:** Reported speech **V:** Meetings and summaries
W22 Mar 8–14	Refactor the capstone code: typing + tests	Cost control: Cost Explorer, tags, budget review	Agent loop: plan → call tool → observe	web3.py: listen to events and logs	AI Guardrails: input validation, output checks, limits	BC Python CLI for your contract (read + write)	Deploy the web3 CLI's API endpoint with SAM; cost check	1-D DP II: Decode Ways, Coin Change, Longest Increasing Subsequence	**G:** Phrasal verbs: *set up, roll out, back up, figure out* **V:** Teamwork and collaboration
W23 Mar 15–21	Library v1.0: README, semantic versioning, changelog	Security review: IAM least-privilege audit, S3 Block Public Access	Cost and latency: caching, smaller models, streaming	Expose the contract through an API endpoint	Py Integration tests for the capstones	AWS Final deploy + simple load test of the capstone	End-to-end test of all capstones	Intervals + Greedy: Merge Intervals, Non-overlapping Intervals, Maximum Subarray, Jump Game	**G:** Polite language and softeners: *Could you…? I was wondering if…* **V:** Job interviews: "Tell me about yourself"
W24 Mar 22–28	Final review: polish Python repos and notes	Clean up resources; architecture diagram + README for the stack	Ship the "Docs Assistant" API on AWS	Wrap-up: README and architecture for the web3 project	AI Demo script + README for the AI capstone	BC Demo and reflection; next steps (e.g. Layer 2s)	**Checkpoint 3:** final demo, retrospective, plan the next 6 months	Review weak spots: re-solve 6 flagged problems against the clock	**G:** Review of all grammar points + presentation signposting (*First, let me show…*) **V:** Demos and presentations

6. Milestone Projects & Free Resources per Track

Python

Milestones

- **Months 1–2:** refactor one of your APIs: full type hints (mypy clean), a custom *@retry* or *@timed* decorator, a generator-based data export.
- **Months 3–4:** add async endpoints and a pytest suite with 80%+ coverage.
- **Months 5–6:** package a reusable utility library with *pyproject.toml* and publish it to TestPyPI.

Resources: Official Python docs (docs.python.org/3: *typing*, *asyncio*, *contextlib*, *functools*) · pytest docs (docs.pytest.org) · Python Packaging User Guide (packaging.python.org)

AI

Milestones

- **Months 1–2:** a scikit-learn classifier on a public dataset, with a metrics report.
- **Months 3–4:** a small PyTorch or fast.ai model trained on your own data; a script that calls an LLM API and returns structured JSON.
- **Months 5–6:** "Docs Assistant": a RAG app over your API docs or study notes, with one agent tool, served as an API and deployed on AWS.

Resources: Andrew Ng's Machine Learning Specialization (DeepLearning.AI / Coursera, audit) and the free DeepLearning.AI Short Courses · fast.ai Practical Deep Learning for Coders (course.fast.ai) · scikit-learn User Guide (scikit-learn.org) + 3Blue1Brown "Neural Networks" videos

AWS

Milestones

- **Months 1–2:** your API running on an EC2 instance with a least-privilege IAM role; a *boto3* script that manages S3 files.
- **Months 3–4:** the API on Lambda + API Gateway + DynamoDB with CloudWatch logs and an architecture diagram; **AWS Certified Cloud Practitioner** passed.
- **Months 5–6:** the whole stack defined as code (SAM or CloudFormation), hosting the AI capstone.

Resources: AWS Skill Builder: "AWS Cloud Practitioner Essentials" + official practice questions (skillbuilder.aws) · AWS Documentation and the official exam guide (docs.aws.amazon.com) · freeCodeCamp AWS Cloud Practitioner full course (YouTube)

Blockchain

Milestones

- **Months 1–2:** a toy blockchain in Python (blocks, hashing, simple proof of work); first CryptoZombies lessons completed.
- **Months 3–4:** your own Solidity contract (e.g. a tip jar or "study log") written and tested in Remix.
- **Months 5–6:** the contract deployed to a testnet, plus a Python CLI or API endpoint built with *web3.py* that reads and writes to it.

Resources: CryptoZombies, interactive Solidity course (cryptozombies.io) · ethereum.org Developer Docs (ethereum.org/developers) + Solidity docs (docs.soliditylang.org) · web3.py documentation (web3py.readthedocs.io)

English

Milestones

- **Months 1–2:** 90+ words in your deck; 8 grammar points practiced; all notes and commits written in English.
- **Months 3–4:** 180+ words; a clear English README for every project; a 2-minute weekly recording every Sunday.
- **Months 5–6:** 270+ words; mock "Tell me about yourself" interview; a 2-minute recorded demo of your capstone, explained in English.

Resources: BBC Learning English, incl. *6 Minute English* (bbc.co.uk/learningenglish) · British Council LearnEnglish grammar lessons (learnenglish.britishcouncil.org) · Cambridge Dictionary (dictionary.cambridge.org) · Write & Improve for writing feedback (writeandimprove.com) · Anki flashcards (apps.ankiweb.net)

LeetCode

Milestones

- **Months 1–2:** ~50 problems; easy ones solved in under 15 min.
- **Months 3–4:** ~100 total; first mediums solved on your own.
- **Months 5–6:** ~150 total (incl. Sunday re-solves); re-solve old problems faster, mostly mediums.

Resources: NeetCode 150 list and video explanations (neetcode.io) · LeetCode (leetcode.com) · Tip: write each pattern in one English sentence (coding + English practice at once)